Convertor — Review and Roadmap

29 September 2026 · build 1.0.0 (aarch64) · reviewed by Claude and Codex independently

Summary

The app works. Conversion is broad and correct, the desktop packaging is real, and the pack system installs and removes cleanly. What follows is what is wrong with it, in the order it is worth fixing.

Two reviews were run over the same commit. Codex read the engine statically and produced ten line-specific findings; Claude measured the running application and the rendered interface. They overlapped on almost nothing, which is the useful outcome: static reading found races and escape paths, measurement found cost and absent controls.

Area	State
Conversion engine	Broad and correct. 185 formats, 33 converters, FFmpeg capability-discovered.
Desktop packaging	Works. 186 MB disk image, launches, converts, no orphan process.
Engine packs	Install, verify and remove correctly from upstream sources.
Downloader	Works, now reports real qualities and sizes. Least mature area.
Job runner	**Has a defect that strands jobs permanently.** See P0-1.
Windows	Never built. CI is written and unverified.

P0 — Correctness, before any new feature

P0-1 · A job can be stranded at "running" forever — *fixed*

engine/app/jobs.py:320 — verified empirically.

`_resolve_inputs()` and `require_route()` run at body line 9 of `_run_item`; its `try` begins at line 23. `guarded()` has no handler, and `_run` awaits with `asyncio.gather` and no outer handler. An exception from either call escapes every handler, `gather` re-raises, and the finalisation that marks the job finished never executes.

Trigger: delete an input file between creating a job and it starting. The item is already marked running, so the item and its job stay running for the life of the process. Nothing times out and nothing reports it.

Fixed. Both calls moved inside the handler and `guarded()` now catches, so one item's failure cannot end the batch. An item whose input is gone fails with "One of the files is no longer available" instead of stranding its job.

P0-2 · Two installs of one pack corrupt each other — *fixed*

engine/app/installer.py:560 — found by Codex, static.

The "is one already running" check and the insertion of the new install record sit in two separate critical sections. Two requests can both pass the check, both start threads, and both manipulate the same pack root and its `.replacing` sibling. One thread can delete the other's verified payload.

Fixed. The id is claimed inside one critical section before any validation or filesystem work, and released if validation then fails. Race-tested with ten concurrent installs of one pack: exactly one worker thread, clean completion, no leaked state.

P0-3 · A failed rename leaves the previous pack displaced — *fixed*

engine/app/installer.py:480 — found by Codex, static.

An install moves the existing pack to `<root>.replacing` and then renames staging into place. If the second rename fails, the working pack is gone from its expected path and nothing restores it. The install reports failure; the user loses a pack that was fine.

Fixed. The retired directory is put back before the error is re-raised, and only deleted once the replacement is in place.

P0-4 · A cancelled download can report success

engine/app/downloader.py:391 — found by Codex, static.

Cancellation is only observed inside the progress hook. A cancel that arrives during post-processing, after the final hook, is never seen, and the worker records `completed`.

Fix: re-check the cancel flag immediately before recording completion.

P1 — Resource discipline

P1-1 · Every status poll walks 25,000 files

engine/app/packs.py — measured.

Endpoint	Cost per call
/api/formats	1 ms
/api/files	1 ms
/api/packs	183 ms
/api/system	201 ms

`Pack.disk_megabytes` walks the entire pack tree to total its size, and `/api/system` includes it. With the office and pdfword packs installed that is **25,304 files walked on every call**, and the interface polls this endpoint. Checking installed-ness costs 0.4 ms; measuring size costs 184 ms.

Fix: compute size only for the packs screen, or cache it and invalidate on install and removal. The header

does not need it at all.

P1-2 · Completed jobs and downloads are never released

jobs.py:179, downloader.py:173 — found by Codex, static.

Both keep every record for the life of the process, with routes, items, warnings, output metadata and listener queues attached. `forget()` exists for jobs and is never called. A long desktop session grows without bound.

P1-3 · Concurrency is bounded per job, not globally

jobs.py:307 — found by Codex, static.

Each job builds its own semaphore, so N jobs permit N × concurrency converters at once. Each job looks bounded; the machine is not.

P2 — Interface

Measured by rendering every page headlessly at 1280, 900 and 390 px with hostile content — a 105-character filename, unicode, an emoji. Findings were checked against screenshots, which is how the worst one was found.

Finding	State
Download screen rendered no quality buttons until a fetch resolved	Fixed
Two-line control used a fixed-height single-row class; labels clipped	Fixed
Fifth nav item clipped the wordmark at phone width	Fixed
Conversion, formats, history, settings at three widths	No clipping or overlap
Nav links are 24 px tall	Deliberate desktop density; small for touch

The lesson worth keeping: a numeric layout probe reported the download screen clean while its entire control group was missing. Every check it ran inspected rendered elements, and the bug was elements that never rendered. A screenshot found it in ten seconds.

Roadmap

Deliberately short. The app's problem is not a shortage of features.

Now — finish what exists

1. **P0-4, the last open defect.** A cancelled download can still report success. P0-1, P0-2 and P0-3 are fixed and verified.

2. **P1-1, the status poll.** One afternoon, and it removes a constant disk cost.
3. **Run the Windows CI once.** It has never executed. Until it does, Windows support is a claim.
4. **A real end-to-end download test in CI.** The "reports success, downloads nothing" defect shipped because every test used a local file server.

Next

1. **Resumable pack downloads.** A 298 MB LibreOffice pack restarts from zero on any interruption, which on a slow mirror is a real loss.
2. **Prune completed jobs and downloads** (P1-2), and cap concurrency globally (P1-3).

Signing — declined, and what replaces it

Apple charges $99 a year for the certificate that would silence Gatekeeper. For a free project distributed on GitHub that is a bad trade, so it is not happening. There is no free path to notarisation: a free Apple developer account cannot notarise, and ad-hoc signing does not satisfy Gatekeeper.

What signing would have bought splits into two things, and one of them is free:

What signing gives	Free substitute
Proof the file is unaltered	A SHA-256 published beside every artefact
Proof of who built it	GitHub build provenance — cryptographic proof the binary came from this repository and commit, verifiable with `gh attestation verify`
No warning on first launch	**Nothing. This one cannot be bought for free.**

So the release notes state the warning plainly rather than pretending it away. On macOS 15 and later — this machine runs 26 — right-click → Open no longer works; the app must be allowed from **System Settings → Privacy & Security → Open Anyway**. Windows shows SmartScreen until the download builds reputation, which it may never do at low volume.

All of this is now in the CI workflow: a release job that only fires when every platform built, checksums for each artefact, build provenance attestation, and install instructions in the release body.

Worth doing, small

1. **A download queue.** Several links, one after another. The engine already supports it; only the screen assumes one.
2. **Remember the last output folder** per screen, rather than defaulting every time.
3. **Show estimated size before conversion** for media, the way the downloader now does. The same surprise exists on the convert screen.

Explicitly not now

Cloud sync, accounts, a plugin system, mobile, batch scripting, a CLI. Each is defensible and none is the

reason this app is not yet ready to hand to someone. Adding any of them before the P0 list is finished would be building on a job runner that can strand a job.

How this was reviewed

Method	What it found
Codex, static engine read	10 findings: races, escape paths, unbounded state
Claude, running measurement	Endpoint cost, 25,304-file walk, memory
Headless render, 3 widths	3 layout defects, all fixed
Screenshots	The absent control group the probe could not see
Acceptance suite, 43 checks	Passing against the packaged app

Codex's findings were not taken on trust: its highest-severity claim was reproduced independently before being written down here. The rest are marked with the confidence Codex itself stated.